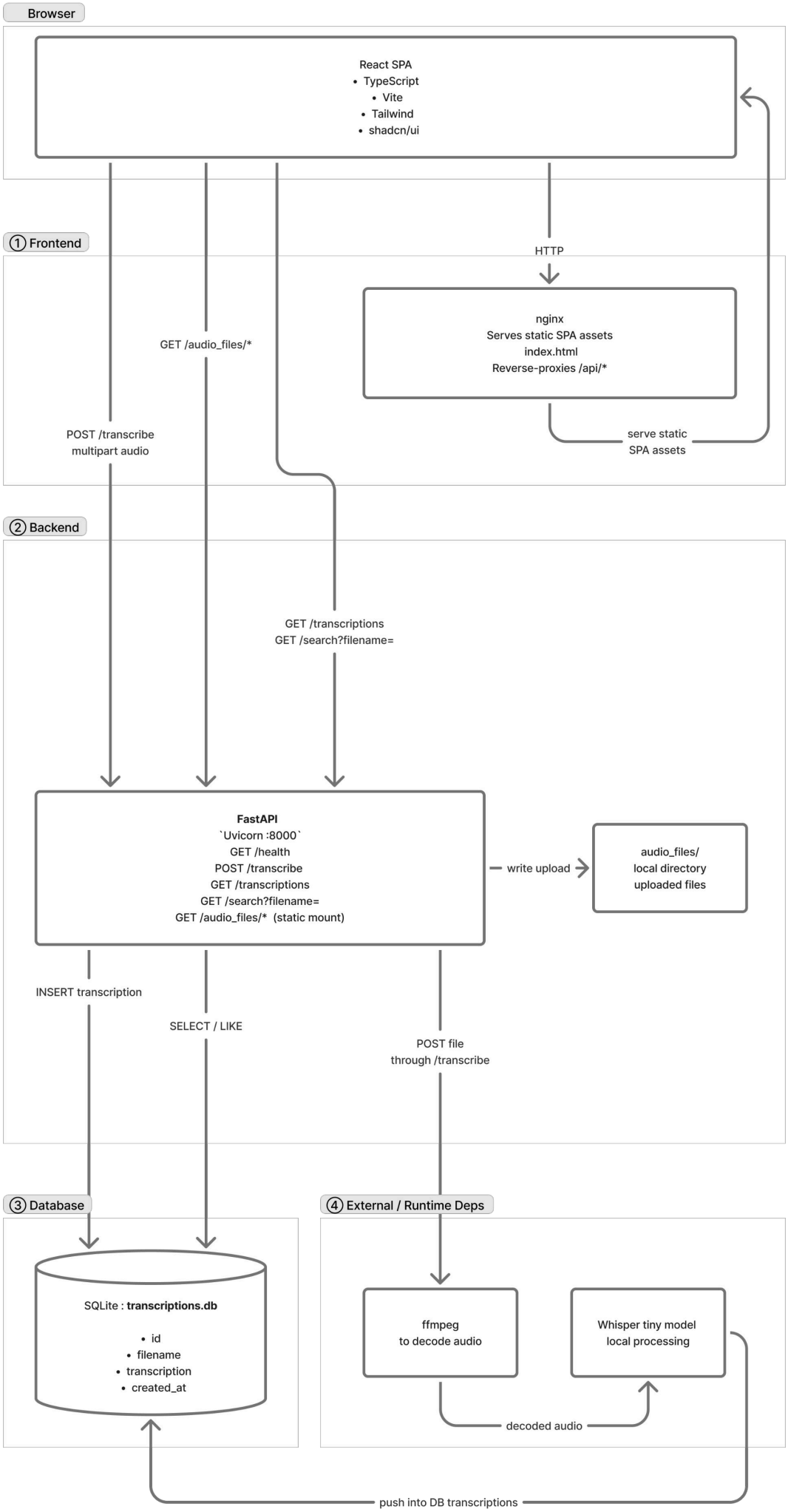
Browser
React SPA
• TypeScript
• Vite
• Tailwind
• shadcn/ui
① Frontend
HTTP
nginx
Serves static SPA assets
index.html
Reverse-proxies /api/*
GET /audio_files/*
POST /transcribe
multipart audio
serve static
SPA assets
② Backend
GET /transcriptions
GET /search?filename=
FastAPI
`Uvicorn :8000`
GET /health
POST /transcribe
GET /transcriptions
GET /search?filename=
GET /audio_files/* (static mount)
write upload
audio_files/
local directory
uploaded files
INSERT transcription
SELECT / LIKE
POST file
through /transcribe
③ Database
SQLite : transcriptions.db
• id
• filename
• transcription
• created_at
④ External / Runtime Deps
ffmpeg
to decode audio
Whisper tiny model
local processing
decoded audio
push into DB transcriptions

Assumptions & considerations

Explanation of the diagram
The frontend is a single-page application built with Vite, Shadcn components and taildwindCSS.

In production, nginx serves those assets and falls back to index.html for client-side routes. The browser sends API requests to the backend base URL configured at build time (VITE_API_BASE_URL), so the frontend and backend can run on different hosts or ports (e.g. nginx on 3000 mapped to 80, API on 8000).

The backend is a FastAPI application. Uploads are written to a local audio_files directory and exposed under /audio_files for playback.

Transcription uses the openai-whisper python package with the tiny model loaded in-process. This is using the local processing. ffmpeg is used to decode the audio formats to parse it for openai-whisper processing.

Results are stored in SQLite (transcriptions.db). Listing and search endpoints read from this database. search uses SQL LIKE on filename.

Assumptions

- Low concurrency and a single-node deployment are acceptable. SQLite and local disk storage fit the stated assessment scope.

- Whisper tiny weights are pre-cached at image build time where Docker is used, avoiding download at container startup.

- CORS is fully open for simplicity in local and assessment environments.

- Uploaded files and the database file live on the same machine as the backend. Did not apply any additional settings for this.

Considerations

- Search is SQL LIKE on filename, not full-text on transcript content.
- No pagination on list endpoints
- Usage of ShadCN + Tailwind components without much modifications for now
  - only added in shadcn presets
- Due to time constraints, only minimal FE work was made. Majority of effort was logic implementation.